NAME:Emmanuel Klbet
REG_NO:CIT-227-043/2025
PROGRAM:Bsc. Software Engineering

Modern Parking Management System

1. Overview and Data Structure Selection

To optimize for performance and ensure fast time complexity for entry and exit operations, the following data structures are utilized:

- **Min-Heap (Priority Queue):** Manages available parking slots. A Min-Heap guarantees that the root node is always the smallest available slot number. This allows assigning the nearest available slot in *O(log N)* time, where N is the number of available slots.
- **Hash Map (Dictionary):** Maps the vehicle's license plate to a record object containing its assigned slot number and entry timestamp. This enables *O(1)* time complexity for vehicle lookups during the exit process.

2. Step-by-Step Algorithm

Phase 1: System Initialization

1. Define the total parking capacity *N*.
2. Populate the Min-Heap with all slot numbers from 1 to *N*.
3. Initialize an empty Hash Map to track actively parked vehicles.
4. Establish a connection to the relational database and initialize the ParkingRecords schema if it does not already exist.

Phase 2: Check Availability (O(1) Time)

1. Check the current size (length) of the Min-Heap.
2. If the length is 0, return a "Parking Full" status.
3. Otherwise, return the length as the number of currently available slots.

Phase 3: Vehicle Entry (O(log N) Time)

1. Invoke the Check Availability phase. If the system is full, reject the entry.

2. Record the current system timestamp as entry_time.
3. Extract (pop) the minimum value from the Min-Heap. This value is assigned as the slot_number.
4. Insert a new key-value pair into the Hash Map: license_plate → (slot_number, entry_time).
5. Execute a SQL INSERT statement into the database with the license_plate, slot_number, and entry_time.
6. Generate and return the parking ticket details to the user.

Phase 4: Vehicle Exit (O(log N) Time)

1. Query the Hash Map using the exiting vehicle's license_plate. If not found, return an error (vehicle not in system).
2. Record the current system timestamp as exit_time.
3. Retrieve the entry_time and slot_number from the Hash Map, then immediately delete the entry from the Hash Map to free memory.
4. Insert (push) the freed slot_number back into the Min-Heap to make it available for future vehicles.
5. Calculate the total parking duration (Exit Time - Entry Time).
6. Invoke the Fee Calculation module to determine the fee_paid.
7. Execute a SQL UPDATE statement to populate the exit_time and fee_paid fields for the corresponding database record.
8. Generate the final receipt.

Phase 5: Tiered Fee Calculation

1. Convert the calculated duration into hours, rounding up to the nearest whole hour.
2. **Tier 1 (Base Rate):** If duration ≤ 1 hour, the fee is a flat base rate (e.g., 50 KES).
3. **Tier 2 (Extended Rate):** If 1 < duration ≤ 3 hours, the fee is 50 KES + (hours - 1) * 30 KES.
4. **Tier 3 (Maximum Rate):** If duration > 3 hours, apply a maximum daily cap or continue scaling linearly depending on business rules.

3. Database Design

A relational database (e.g., SQLite, PostgreSQL) is used to track historical and active parking

sessions. A single core table handles both active states (where exit data is NULL) and historical transactions.

Table: ParkingRecords

Column Name	Data Type	Constraints	Description
record_id	Integer	Primary Key, Auto-Increment	Unique identifier for the parking transaction.
license_plate	Varchar(20)	Not Null	The vehicle's registration plate.
slot_number	Integer	Not Null	The specific parking slot assigned to the vehicle.
entry_time	DateTime	Not Null	The exact timestamp when the vehicle entered.
exit_time	DateTime	Nullable	The timestamp when the vehicle exited (NULL if actively parked).
fee_paid	Decimal/Float	Nullable	The calculated amount paid upon exit (NULL if actively parked).